# Callbacks

10 min read  Updated 24 Jul 2026  Topic 16 of 70



## Why it matters

Node.js is single-threaded for your JavaScript, yet it powers servers that juggle thousands of simultaneous connections. It pulls that off by never blocking the one thread on slow work — reading a file, querying a database, calling another service. Instead of freezing until the result is ready, Node takes a function from you, walks away to do the slow work elsewhere, and calls your function back when the answer arrives. That function is a **callback**, and it is the original, foundational async pattern in Node. Promises and `async/await` are built on top of it.

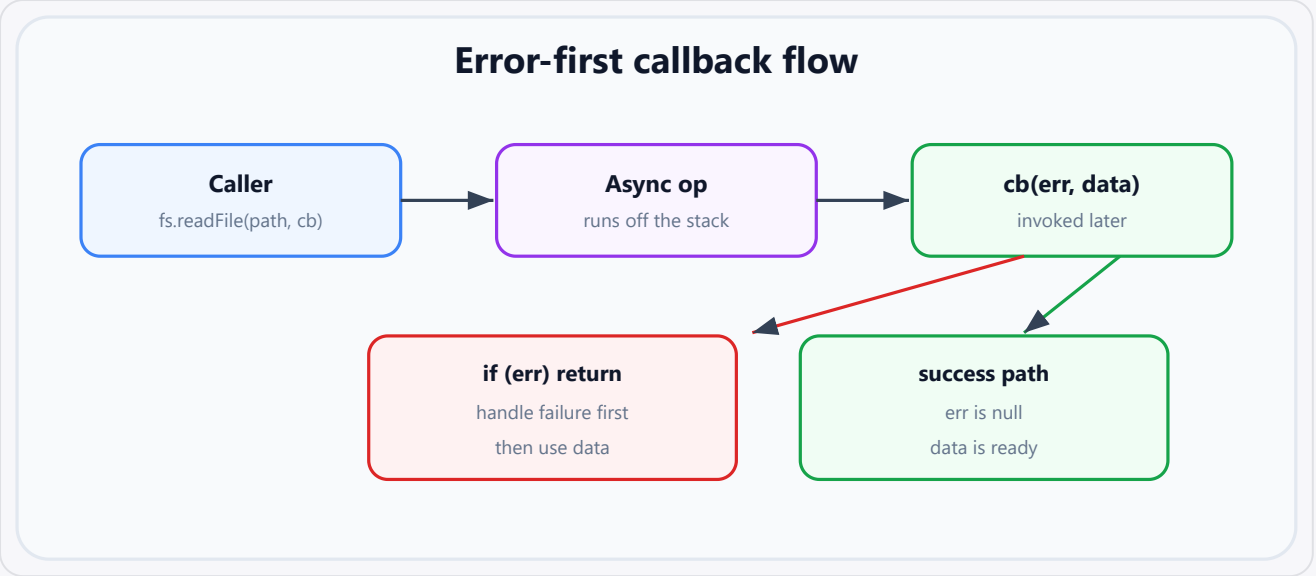
You cannot really understand Node without understanding callbacks. Every higher-level abstraction — the entire streams system, the `EventEmitter`, most of the classic `fs` and `net` APIs, and countless npm packages written before 2016 — speaks callbacks. When you debug a stack trace that jumps out of nowhere, or wrap an old library so it returns a promise, you are working directly with this pattern. Interviewers reach for it constantly because it exposes whether you actually understand the event loop or just memorised `await`.

## The concept, simply

A callback is just a function you pass to another function, to be invoked *later*, when some work finishes. Synchronous callbacks (like the function you give `Array.map`) run immediately. The interesting ones are **asynchronous** callbacks: you hand them over now, your code keeps running, and Node invokes them on a future turn of the event loop once the awaited event happens.

Node standardised a convention called the **error-first callback** (sometimes "node-style" or "errback"): the callback's first parameter is an error, and the actual result comes after

it. If the operation failed, the first argument is an `Error` and the rest are undefined; if it succeeded, the first argument is `null` and the data follows. This single convention makes thousands of unrelated APIs feel consistent.




In an error-first callback the failure branch is always checked first; only then is the data used.

## How it works

When you call `fs.readFile('a.txt', cb)`, Node does not read the file on your thread. It registers the request with the operating system (via the libuv thread pool for file I/O) and returns immediately — the very next line of your code runs before the file is read. Your callback is parked. When the read completes, libuv places your callback on a queue, and once the current call stack is empty the event loop picks it up and runs it. This is why a `console.log` after the `readFile` call prints *before* the callback fires.

The critical consequence: a callback runs on a **fresh, empty call stack**, disconnected from the code that scheduled it. The function that called `readFile` has long since returned. That is why you cannot wrap an async callback in a `try/catch` and expect to catch its error — by the time the callback throws, the `try` block is gone from the stack. The error-first convention exists precisely because `throw` is useless here: errors must be passed *as data* into the callback instead.

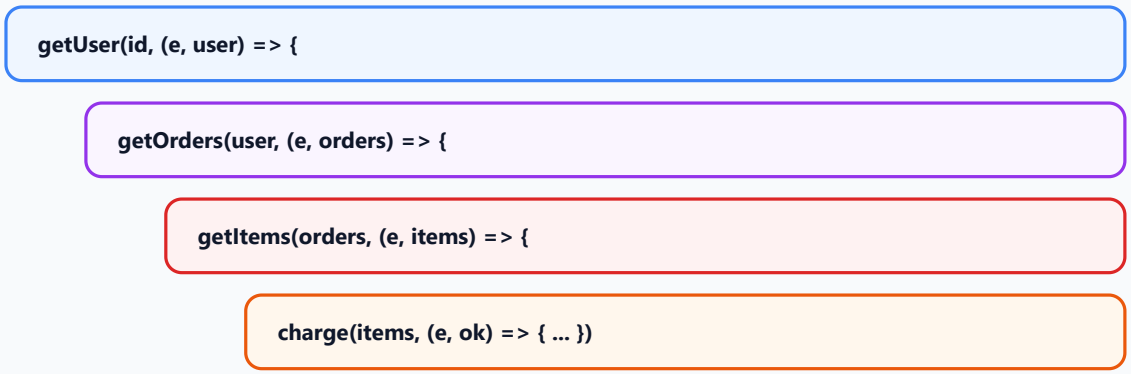
Because each async step schedules the next one only from inside its own callback, sequencing several dependent operations means nesting callbacks inside callbacks. Each level must independently check its own error argument. This is where the pattern's

There's a subtle timing distinction worth internalising. Not all deferrals are equal: `process.nextTick(cb)` runs your callback *before* the event loop continues to the next phase — effectively at the front of the queue, even ahead of promise microtasks — whereas `setImmediate(cb)` runs it on the next loop iteration's check phase, and `setTimeout(cb, 0)` runs it in the timers phase. For merely making a synchronous path asynchronous, `setImmediate` is the safest default because `nextTick` can starve the event loop if you schedule it recursively. The takeaway for callbacks specifically: when you need to defer to avoid releasing Zalgo, prefer `setImmediate` unless you have a concrete reason to jump the queue.

The three ways to defer a callback differ in *when* on the loop they run, and confusing them causes ordering bugs that only appear under load:

| Primitive | Runs | Drains |
| --- | --- | --- |
| `process.nextTick(cb)` | Before the loop advances, ahead of promise microtasks | Fully, af |
| Promise `.then` / `queueMicrotask` | After nextTick, before the next phase | Fully, be |
| `setImmediate(cb)` | Next iteration, in the check phase | One pe |
| `setTimeout(cb, 0)` | Timers phase, after a ≥1ms clamp | One pe |

One more mechanic behind file-system callbacks explains a lot of otherwise-baffling throughput ceilings. Network sockets use the operating system's own event notification, so thousands can be in flight at once. File I/O and a few CPU-bound operations (`crypto.pbkdf2`, `zlib`, DNS `lookup`) cannot be done that way, so libuv runs them on a small pool of worker threads whose default size is only **four** (`UV_THREADPOOL_SIZE`). Fire off five concurrent `fs.readFile` calls and the fifth callback waits for a worker to free up, even though your JavaScript looks fully parallel. Knowing which of your callbacks share that pool tells you which ones can actually overlap and which merely queue.




Each nested level repeats its own if (e) check — the pyramid of doom.

Dependent async steps nest, each repeating its own error check — the pyramid of doom.

## Code examples

The canonical error-first pattern. Notice the error branch is handled first and returns early, so the success code never runs on a failure:

JAVASCRIPT Copy

```
const fs = require('node:fs');

fs.readFile('config.json', 'utf8', (err, data) => {
  if (err) {
    console.error('Failed to read config:', err.message);
    return; // stop — do NOT fall through to use data
  }
  const config = JSON.parse(data);
  console.log('Port:', config.port);
});

console.log('This logs FIRST — readFile is async.');
```

Sequencing dependent steps produces the nesting the diagram showed. Each step needs the previous step's result:

JAVASCRIPT Copy

```
  getOrders(user.id, (err, orders) => {
    if (err) return done(err);
    getItems(orders[0].id, (err, items) => {
      if (err) return done(err);
      done(null, items); // finally, the result
    });
  });
});
```

Writing an async function of your own means *always* calling the callback the same way — error first, and never both calling back with an error and continuing. Deferring with `setImmediate` keeps the API consistently asynchronous:

JAVASCRIPT Copy

```
function loadUser(id, cb) {
  if (typeof id !== 'number') {
    // Defer so the callback never runs synchronously — see gotchas below.
    return setImmediate(() => cb(new Error('id must be a number')));
  }
  db.find(id, (err, row) => {
    if (err) return cb(err);        // pass the error along
    cb(null, { id: row.id, name: row.name }); // null error on success
  });
}
```

Running independent callback operations in parallel and collecting results by hand shows why libraries and promises were welcomed. You must count completions yourself and guard against calling `done` twice:

JAVASCRIPT Copy

```
function loadAll(ids, done) {
  const results = [];
  let remaining = ids.length;
  let finished = false;
  if (remaining === 0) return done(null, results);
```

```
      if (err) { finished = true; return done(err); }
      results[i] = user;            // preserve order by index
      if (--remaining === 0) done(null, results);
    });
  });
}
```

## Common mistakes & gotchas

- **Wrapping an async callback in try/catch.** The `try` block has already unwound by the time the callback runs, so it catches nothing. Handle the `err` argument instead.
- **Forgetting to `return` after handling the error.** Without the early return, execution falls through and runs the success path with undefined data — often crashing on `JSON.parse(undefined)` or a null dereference.
- **Calling the callback more than once.** Firing `cb` on error and then again on success (or in a loop) causes duplicated downstream work and double responses. Guard with a `finished` flag or return after each call.
- **Releasing Zalgo — a callback that is sometimes sync, sometimes async.** If `loadUser` called `cb` synchronously for the validation error but asynchronously for the DB path, callers get inconsistent ordering and subtle bugs. Always defer synchronous callbacks with `setImmediate` or `process.nextTick`.
- **Swallowing errors.** An empty `if (err) {}` or an ignored first argument hides real failures. At minimum log and stop.
- **Losing `this`.** Passing a method as a bare callback (`emitter.on('x', obj.handle)`) detaches `this`. Use an arrow wrapper or `.bind()`.
- **Assuming `fs` callbacks run in parallel.** They share libuv's four-thread pool by default, so heavy concurrent file, crypto, or zlib work serialises behind it. Raise `UV_THREADPOOL_SIZE` (set it before the first pool use) or throttle the work if throughput plateaus.

## Best practices

- Follow the error-first convention rigidly in every callback you author — first arg is the error, and only the error.
- Handle the error branch first and `return` immediately, so the happy path reads linearly below it.
- Keep callbacks async-only: never invoke a callback synchronously; defer with `setImmediate` to avoid Zalgo.
- Name your callback functions instead of deeply nesting anonymous ones — flatter code and better stack traces.
- For anything beyond one or two steps, promote to promises with `util.promisify` or use the promise-based core APIs (`fs.promises`) rather than hand-nesting.
- Never call a callback twice; enforce it with a guard flag when the control flow makes it possible.

## ⭐ Interview lens

**Q: What is an error-first callback and why did Node adopt it?**
Because an async callback runs on a fresh stack, `throw` cannot reach the original caller. Passing the error as the first argument gives every async API a uniform, throw-free way to report failure, so callers always check the same place.

**Q: Why does `console.log` after `fs.readFile` print before the file contents?**
`readFile` offloads the work and returns immediately; the callback is queued and only runs after the current synchronous code finishes and the stack empties. This is the event loop deferring the callback to a later tick.

**Q: Can a `try/catch` catch an error thrown inside an async callback?**
No. The `try` block has already returned by the time the callback fires on a new stack. The thrown error escapes as an uncaught exception. This is the core reason errors are passed as data, not thrown.

Deeply nested callbacks caused by sequencing dependent async steps — each step can only schedule the next from inside its own callback. The trap: it isn't just ugly, it also duplicates error handling at every level and makes early exit awkward.

**Q: What is "releasing Zalgo"? (trap)**

Writing a function that invokes its callback synchronously on one path and asynchronously on another. Callers can't reason about ordering, leading to race-like bugs. Fix by always deferring the synchronous path with `setImmediate` or `process.nextTick`.

**Q: How do you convert a callback API into a promise?**

Wrap it with `util.promisify(fn)` (which relies on the error-first convention), or construct a `new Promise` and call `resolve(data)` / `reject(err)` from inside the callback.

## Cheat-card

- **Shape:** `(err, data) ⇒ {}` — error first, always.
- **Runs:** later, on an empty stack, via the event loop.
- **Errors:** passed as arg #1, never thrown — `try/catch` won't catch them.
- **Rule:** handle `err`, then `return`; never call `cb` twice.
- **Zalgo:** keep callbacks always-async; defer sync paths with `setImmediate`.
- **Escape hatch:** `util.promisify` / `fs.promises` to move up to promises.

QUICK CHECK — no marks, just for you.

1. In Node's error-first callback convention, what is the first parameter?

- ◯ The result data

- A status code
- An error (null on success, an Error on failure), with the result following

Check answer

2. Why can't a `try/catch` around an async callback catch an error thrown inside it?

- The callback runs on a fresh, empty stack after the try block has already unwound
- try/catch is disabled in async functions
- The error is swallowed by libuv
- Callbacks never throw errors

Check answer

3. What is "releasing Zalgo"?

- Blocking the event loop with a synchronous loop
- Writing a function that calls its callback synchronously on one path and asynchronously on another
- Throwing a string instead of an Error
- Calling a callback exactly once

Check answer

## LEARN MORE

### ARTICLES & DOCS

- JavaScript Asynchronous Programming and Callbacks
  nodejs.org
- Introducing asynchronous JavaScript
  developer.mozilla.org

### VIDEOS




- Async JS Crash Course - Callbacks, Promises, Async Await
  Traversy Media




- Error First Callback Patterns in JS
  Steve Griffith - Prof3ssorSt3v3

Hand-picked references for this lesson.